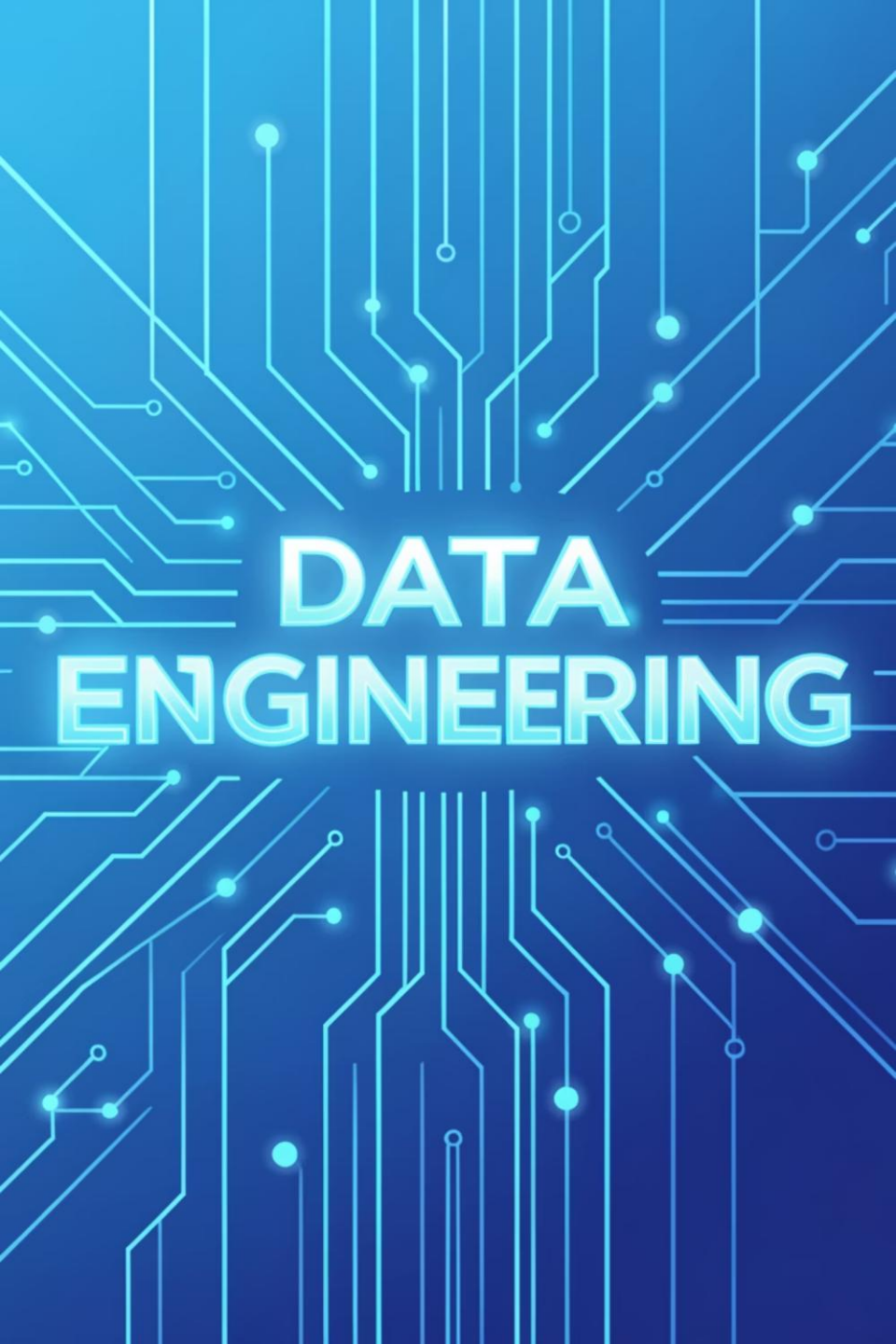




DATA ENGINEERING

Data Engineering Spring '26 – DS471

January 27, 2026

Lecture 4

Logistics: Your Course Essentials

Class Timings

Monday (LH6), Tuesday (LH8): 11:30 AM – 12:20 PM

Friday (LH8): 11:00 AM – 11:50 AM

Office Hours

Monday, Tuesday: 10:00 AM – 11:00 AM, S18 2nd Floor NAB or by appointment.

Credit Hours

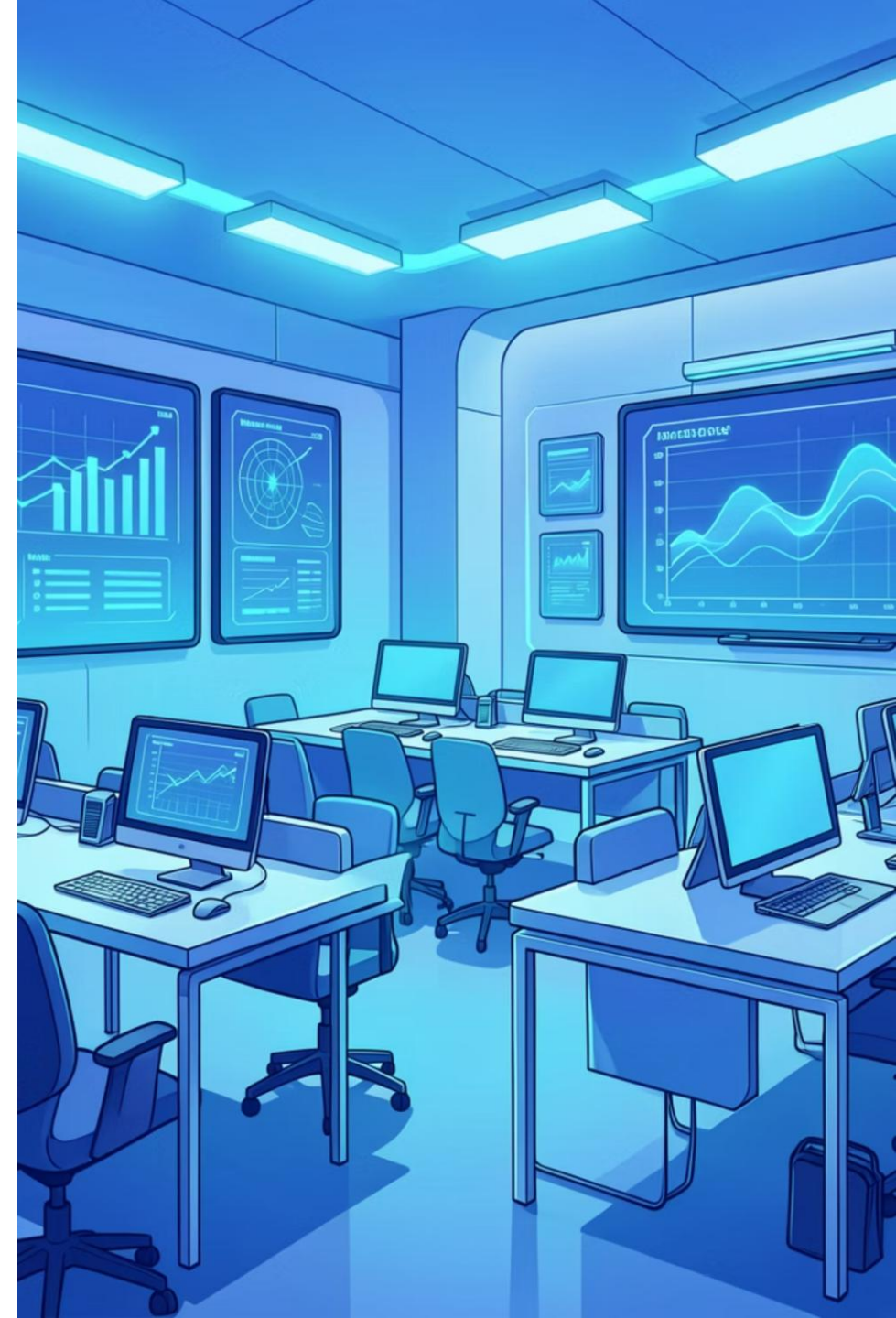
2 hours (Lectures) + 1 hour (Practical Session)

Contact & Support

MS Teams: [TBD](#)

Email: farah.saeed@giki.edu.pk

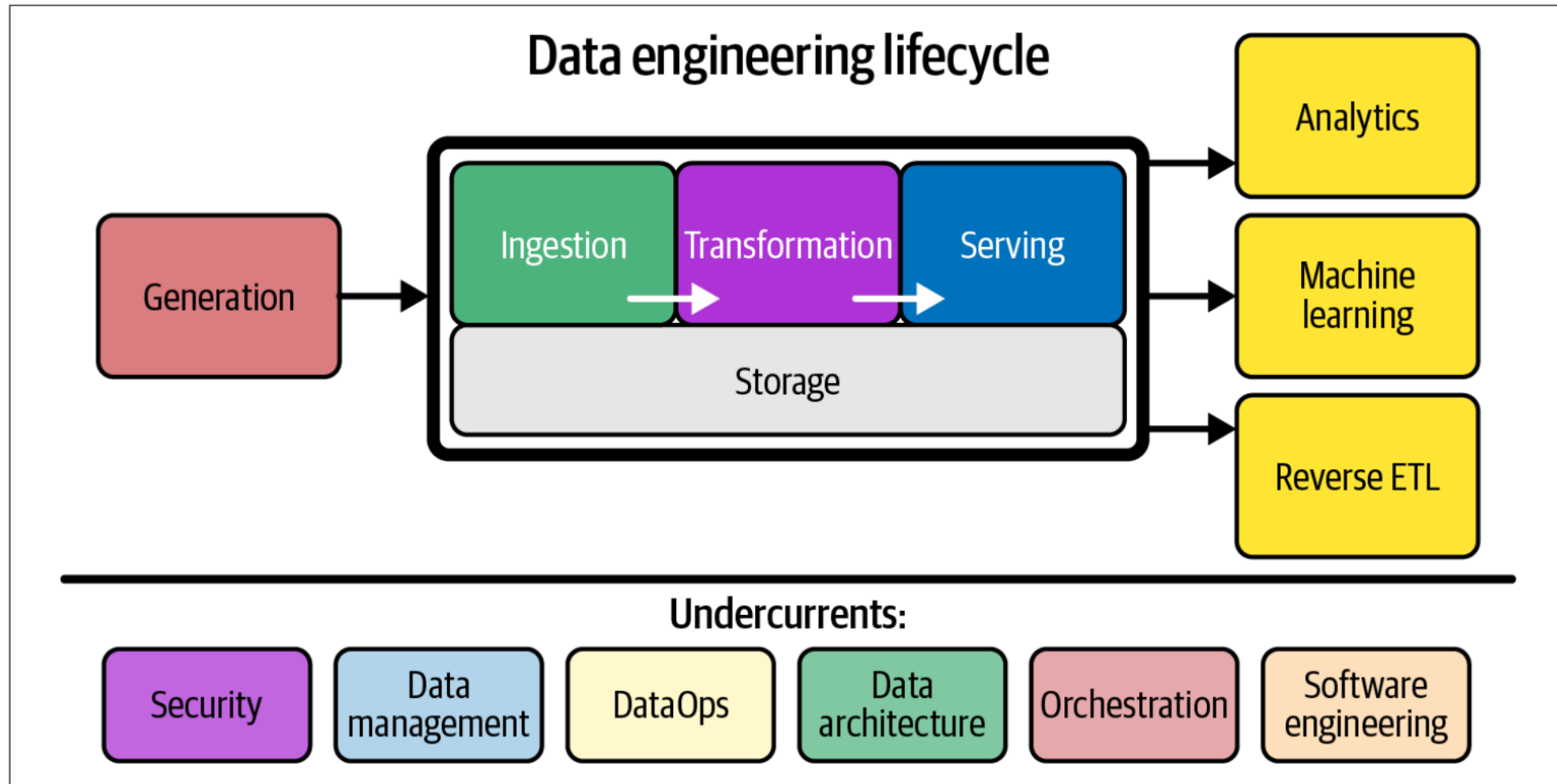
WhatsApp: Reach out for quick queries



Data Engineering Lifecycle

The data engineering lifecycle comprises stages that turn raw data ingredients into a useful end product, ready for consumption by analysts, data scientists, ML engineers, and others.

Technical Responsibilities of data engineer



SQL Python Java or Scala bash and more tools

The Continuum of Data Engineering Roles, from A to B

Type A Data Engineer

Abstraction

Type A data engineers manage the data engineering lifecycle mainly by using entirely off-the-shelf products, managed services, and tools.

Type B Data Engineer

Build

Type B data engineers build data tools and systems that scale and leverage a company's core competency and competitive advantage.

Data Engineers inside an organization

Internal-Facing Versus External-Facing Data Engineers

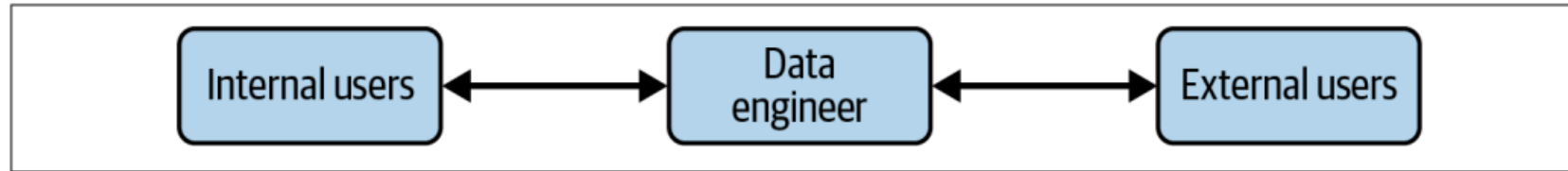


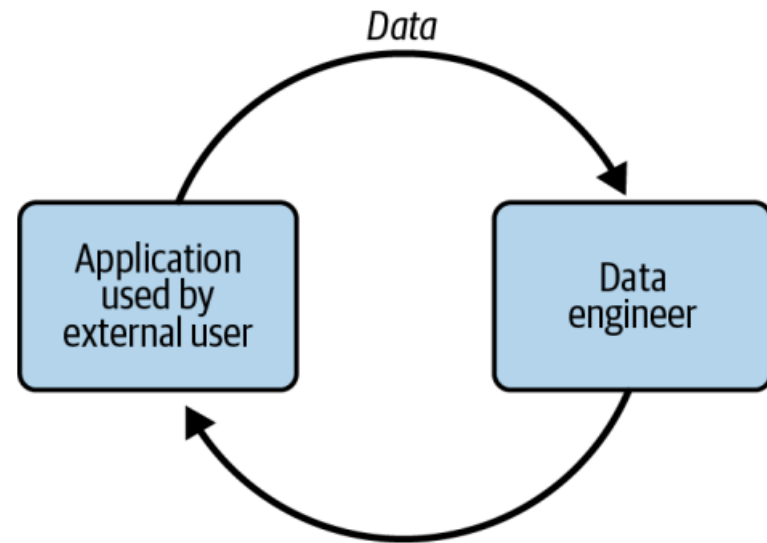
Figure 1-9. The directions a data engineer faces

- An **external-facing** data engineer typically aligns with the users of external-facing applications, such as social media apps, Internet of Things (IoT) devices, and e-commerce platforms
- An **internal-facing** data engineer typically focuses on activities crucial to the needs of the business and internal stakeholders. E.g. creating and maintaining data pipelines and data warehouses for BI dashboards, reports, business processes, data science, and ML models

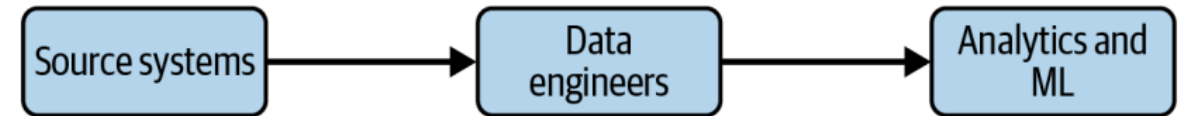
Data Engineers inside an organization

Internal-Facing Versus External-Facing Data Engineers

- **External-facing** data engineer systems



- **Internal-facing** data engineer systems



Data Engineering and other technical roles

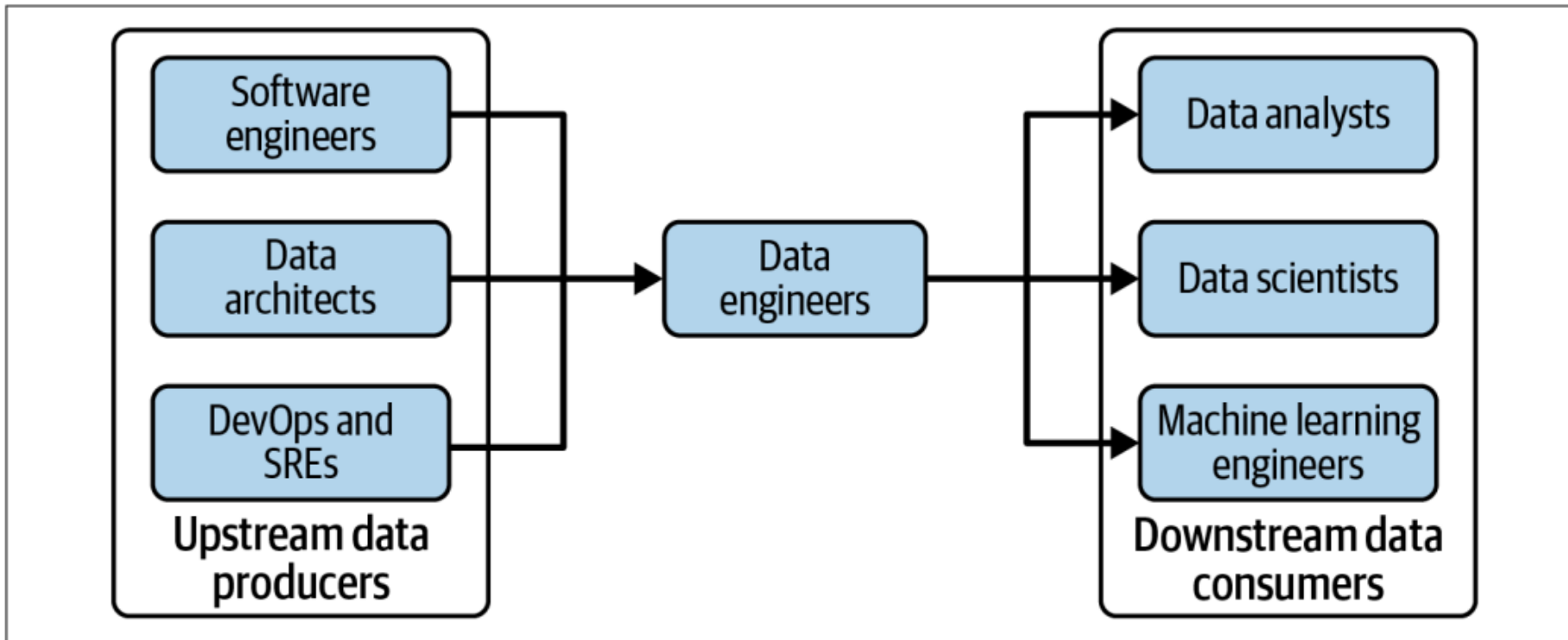
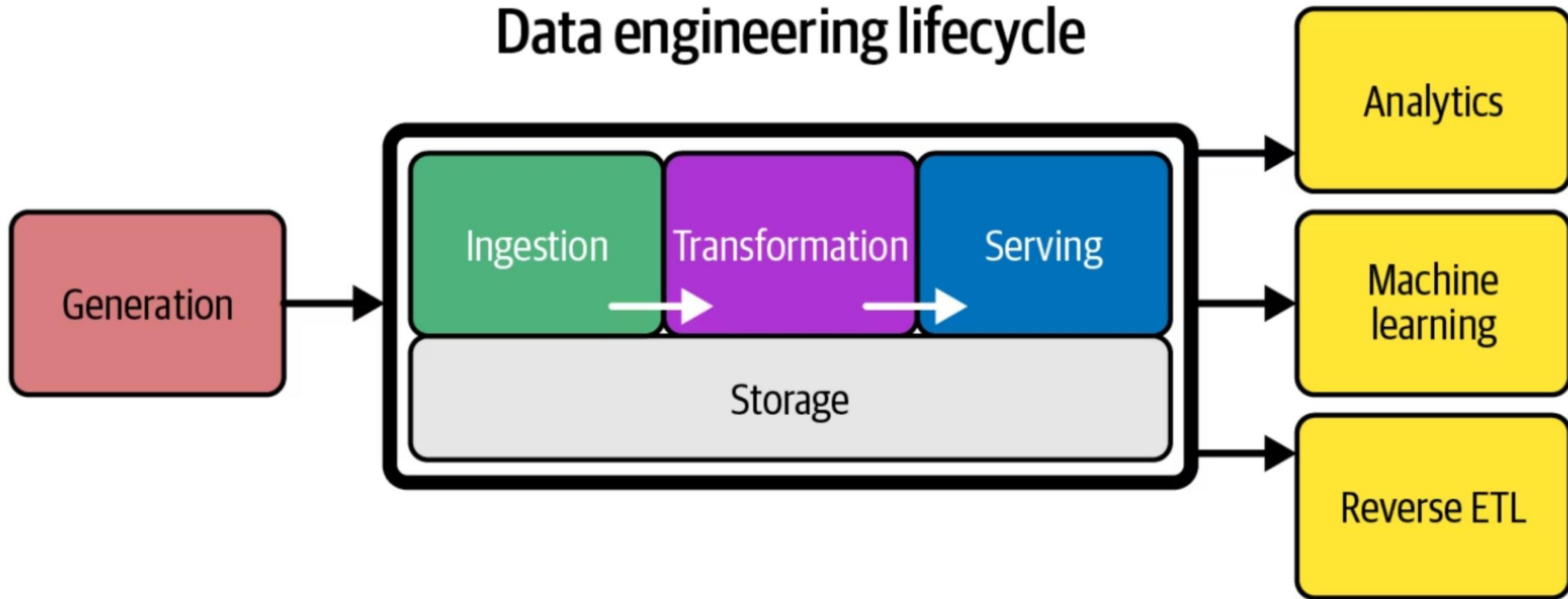
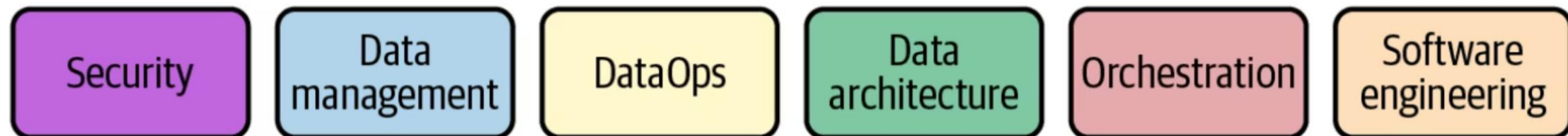


Figure 1-12. Key technical stakeholders of data engineering

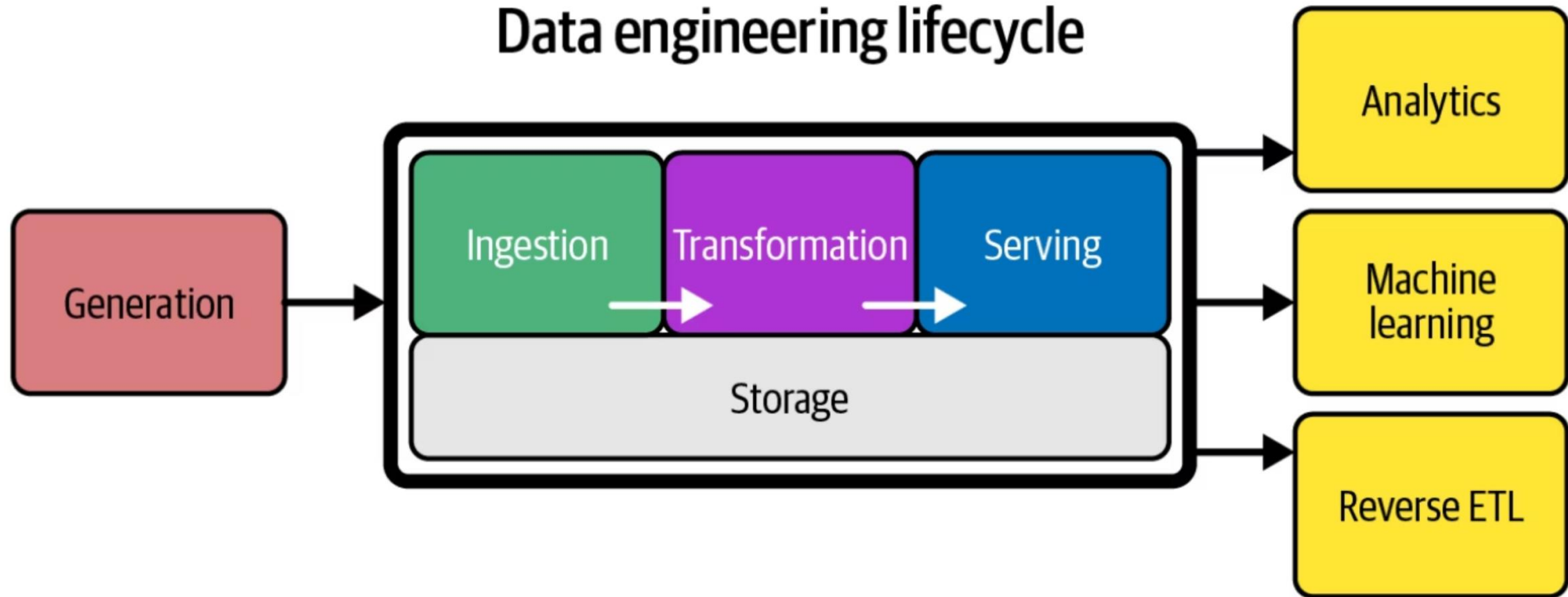
Data engineering lifecycle



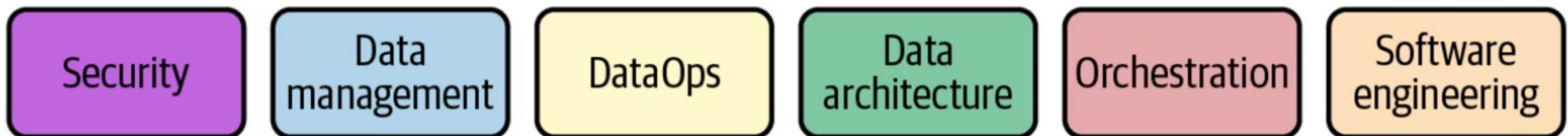
Undercurrents:



Data engineering lifecycle



Undercurrents:



Various stages of the lifecycle may repeat themselves, occur out of order, overlap, or weave together in interesting and unexpected ways.

Data lifecycle vs Data Engineering Lifecycle

Whereas the full data lifecycle encompasses data across its entire lifespan, the data engineering lifecycle

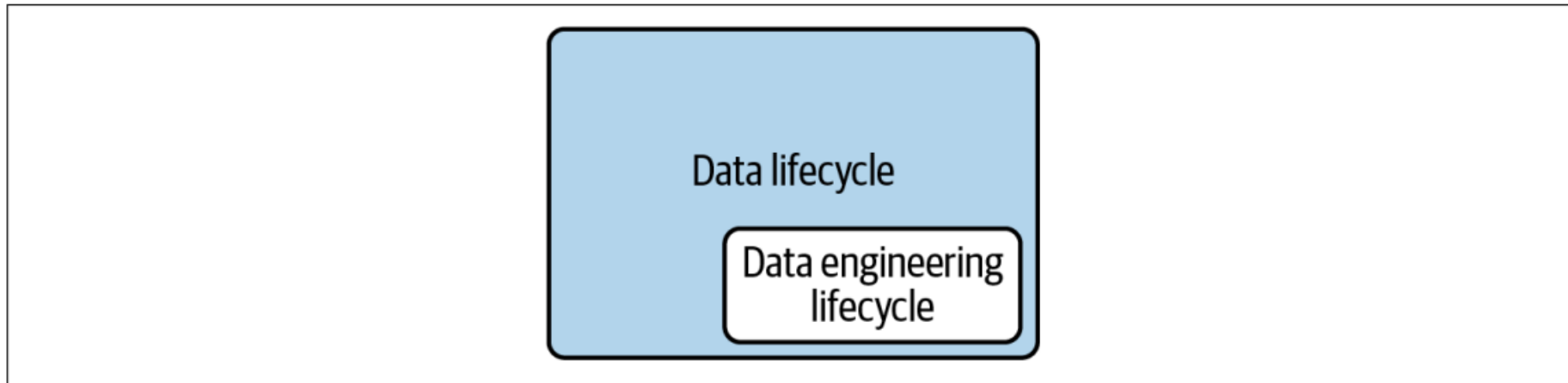


Figure 2-2. The data engineering lifecycle is a subset of the full data lifecycle

Generation: Source Systems

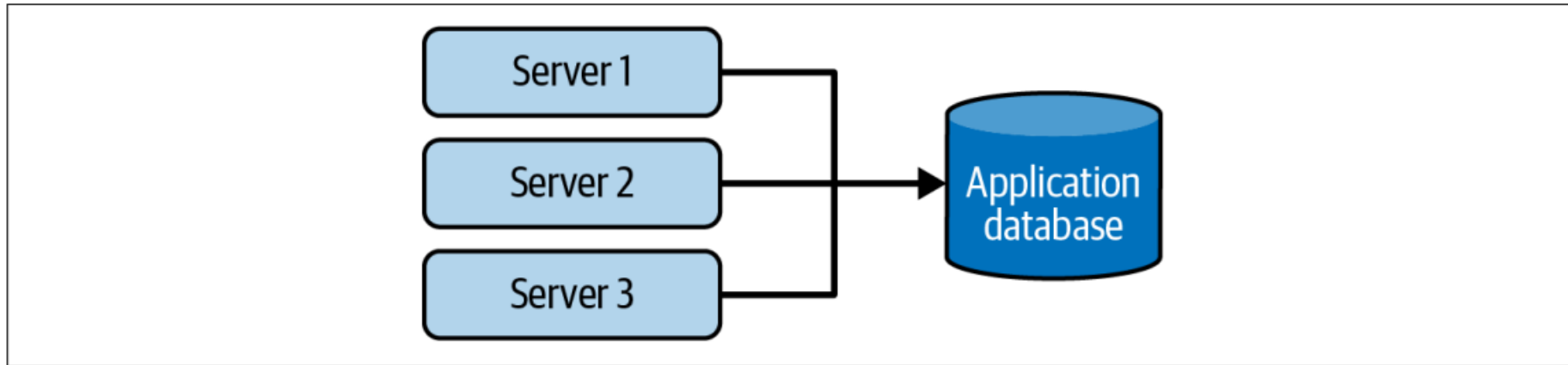


Figure 2-3. Source system example: an application database

Generation: Source Systems

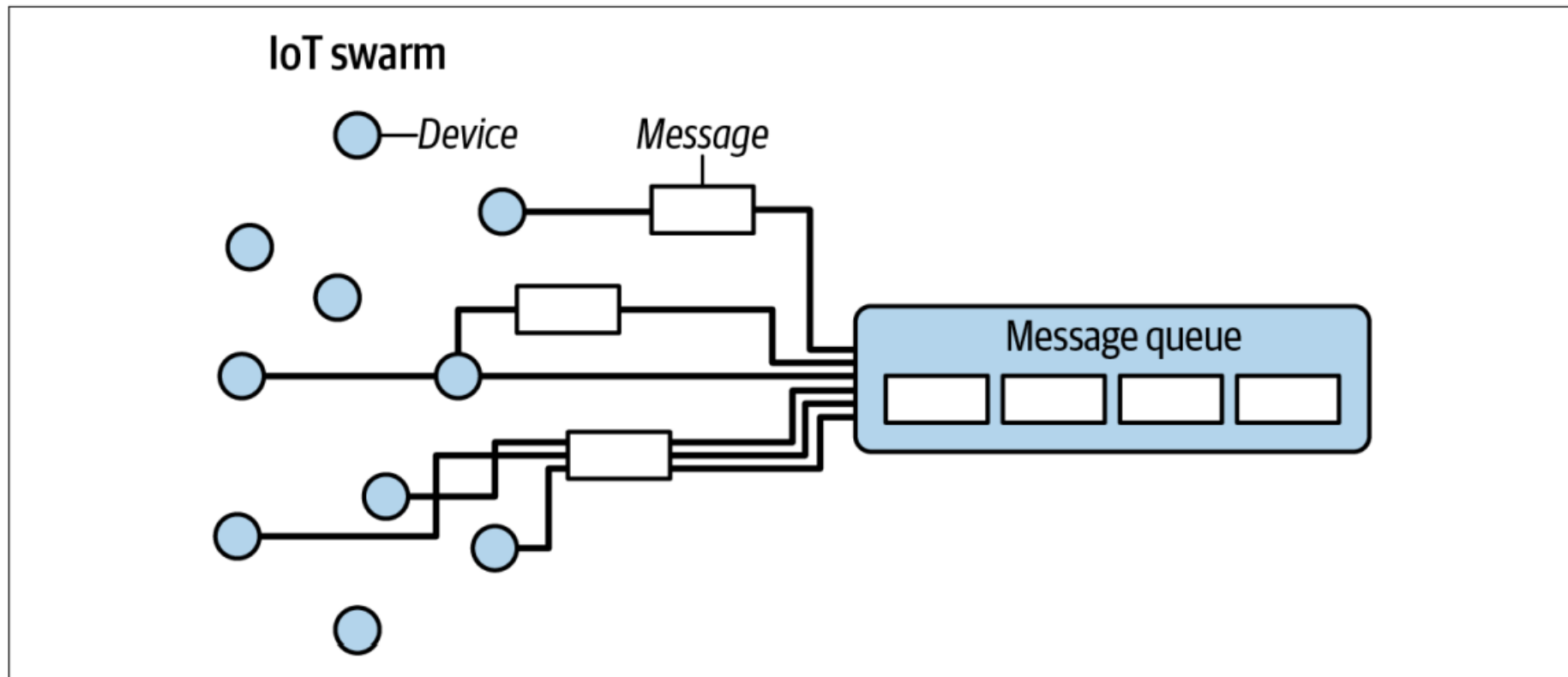


Figure 2-4. Source system example: an IoT swarm and message queue

Generation: Source Systems

Evaluating source systems: Key engineering considerations

- What are the essential characteristics of the data source? Is it an application? A swarm of IoT devices?
- How is data persisted in the source system? Is data persisted long term, or is it temporary and quickly deleted?
- At what rate is data generated? How many events per second? How many giga- bytes per hour?
- What level of consistency can data engineers expect from the output data? If you're running data-quality checks against the output data, how often do data inconsistencies occur—nulls where they aren't expected, lousy formatting, etc.?

Generation: Source Systems

Evaluating source systems: Key engineering considerations

- How often do errors occur?
- Will the data contain duplicates?
- Will some data values arrive late, possibly much later than other messages produced simultaneously?
- What is the schema of the ingested data? Will data engineers need to join across several tables or even several systems to get a complete picture of the data?
- If schema changes (say, a new column is added), how is this dealt with and communicated to downstream stakeholders?

Generation: Source Systems

Evaluating source systems: Key engineering considerations

- How frequently should data be pulled from the source system?
- For stateful systems (e.g., a database tracking customer account information), is data provided as periodic snapshots or update events from change data capture (CDC)?
What's the logic for how changes are performed, and how are these tracked in the source database?
- Who/what is the data provider that will transmit the data for downstream consumption?
- Will reading from a data source impact its performance?
- Does the source system have upstream data dependencies? What are the characteristics of these upstream systems?
- Are data-quality checks in place to check for late or missing data?

Storage

Storage runs across the entire data engineering lifecycle, often occurring in multiple places in a data pipeline, with storage systems crossing over with source systems, ingestion, transformation, and serving.

Storage

Evaluating storage systems: Key engineering considerations

- Is this storage solution compatible with the architecture's required write and read speeds?
- Will storage create a bottleneck for downstream processes?
- Do you understand how this storage technology works? Are you utilizing the storage system optimally or committing unnatural acts? For instance, are you applying a high rate of random access updates in an object storage system? (This is an antipattern with significant performance overhead.)
- Will this storage system handle anticipated future scale? You should consider all capacity limits on the storage system: total available storage, read operation rate, write volume, etc.
- Will downstream users and processes be able to retrieve data in the required service-level agreement (SLA)?

Storage

Evaluating storage systems: Key engineering considerations

- Will downstream users and processes be able to retrieve data in the required service-level agreement (SLA)?
- Are you capturing metadata about schema evolution, data flows, data lineage, and so forth? Metadata has a significant impact on the utility of data. Meta- data represents an investment in the future, dramatically enhancing discoverability and institutional knowledge to streamline future projects and architecture changes.
- Is this a pure storage solution (object storage), or does it support complex query patterns (i.e., a cloud data warehouse)?
- Is the storage system schema-agnostic (object storage)? Flexible schema (Cassan- dra)? Enforced schema (a cloud data warehouse)?
- How are you tracking master data, golden records data quality, and data lineage for data governance?
- How are you handling regulatory compliance and data sovereignty? For example, can you store your data in certain geographical locations but not others?

Storage

Data Access Frequency

- **Hot data** is commonly retrieved many times per day, perhaps even several times per second.
- **Lukewarm data** might be accessed every so often—say, every week or month.
- **Cold data** is seldom queried and is appropriate for storing in an archival system.

Ingestion

Unreliable source and ingestion systems have a ripple effect across the data engineering lifecycle.

Ingestion

Key Engineering Considerations

- What are the use cases for the data I'm ingesting? Can I reuse this data rather than create multiple versions of the same dataset?
- Are the systems generating and ingesting this data reliably, and is the data available when I need it?
- What is the data destination after ingestion?
- How frequently will I need to access the data?
- In what volume will the data typically arrive?
- What format is the data in? Can my downstream storage and transformation systems handle this format?
- Is the source data in good shape for immediate downstream use? If so, for how long, and what may cause it to be unusable?
- If the data is from a streaming source, does it need to be transformed before reaching its destination? Would an in-flight transformation be appropriate, where the data is transformed within the stream itself?

Ingestion

Batch vs Streaming

Batch : Batch ingestion is a specialized and convenient way of processing this stream in large chunks—for example, handling a full day's worth of data in a single batch.

Streaming: Streaming ingestion allows us to provide data to downstream systems—whether other applications, databases, or analytics systems—in a continuous, real-time fashion. Here, real-time (or near real-time) means that the data is available to a downstream system a short time after it is produced (e.g., less than one second later). The latency required to qualify as real-time varies by domain and requirements.

Ingestion

Batch vs Streaming

Key Engineering Considerations

- If I ingest the data in real time, can downstream storage systems handle the rate of data flow?
- Do I need millisecond real-time data ingestion? Or would a micro-batch approach work, accumulating and ingesting data, say, every minute?
- What are my use cases for streaming ingestion? What specific benefits do I realize by implementing streaming? If I get data in real time, what actions can I take on that data that would be an improvement upon batch?
- Will my streaming-first approach cost more in terms of time, money, maintenance, downtime, and opportunity cost than simply doing batch?
- Are my streaming pipeline and system reliable and redundant if infrastructure fails?
- What tools are most appropriate for the use case? Should I use a managed service (Amazon Kinesis, Google Cloud Pub/Sub, Google Cloud Dataflow) or stand up my own instances of Kafka, Flink, Spark, Pulsar, etc.? If I do the latter, who will manage it? What are the costs and trade-offs?
- If I'm deploying an ML model, what benefits do I have with online predictions and possibly continuous training?
- Am I getting data from a live production instance? If so, what's the impact of my ingestion process on this source system?

Ingestion

Push vs Pull Model

In the **push model** of data ingestion, a source system writes data out to a target, whether a database, object store, or filesystem.

In the **pull model**, data is retrieved from the source system.